SNES Assembly Adventure 01: Develop for the SNES

26 Jul 2018

Update January 2023: Mesen 2 has been released, which adds Linux support

Update February 2022: All code examples from all articles in this series can now be found on Github in one repository

Update January 2022: Added build instructions for Mesen-SX emulator. Updated build instructions for bsnes+. Updated instructions to work with MSYS2.

This is the first of several articles on developing games for the Super Nintendo Entertainment System. Retro Gaming has been strong in the past years, with hundreds of new games released. Most aim to re-create the graphic and music style of so-called 8- or 16-bit consoles like The SNES or Sega Genesis/Mega Drive. But how were those classics made that so many of today's games try to emulate? These articles aim to show you the steps for programming your own game for the SNES (and hopefully other consoles and machines in the future). We will cover all topics necessary for writing a SNES game from the beginning to the end. This article will kick-start your project by helping you to set up a development environment. Read on to start your own SNES Assembly Adventure.

There are a few tutorials on programming the SNES on the web, so why write another one? The honest answer is: that I need content for my blog. I'm not an awesome illustrator or artist, so I have to resort to what I'm good at, assembly programming. Hence, I will write about programming in assembly for the SNES. Many game devs use tumblr or other blogs to promote their indie projects or for people to show their (pixel) art and mockups. I hope I can show you how to turn it into an actual SNES game. Not a 16-bit style game, an actual SNES 16-bit game that runs to the console as well as in an emulator.

Who is this for?

I will assume that you have no prior experience in machine code/assembly programming. Also, you need no to little knowledge of how a CPU works as we will cover this. But you should be

familiar with some basic higher-level programming concepts like functions, variables, pointers, etc. If you have a general idea of what those terms mean, you will be able to follow this series. Moreover, you should be familiar with your system's command line and know how to invoke programs with it. Finally, a basic understanding of the binary and hexadecimal numbering system is necessary - but I will give you a quick refresher in the next article.

Content

Before we start, I want to outline the contents I will cover in these articles. In this article we start with setting up a development environment; your playground to create SNES games. The next couple of articles will introduce you to the 65816 microprocessor and how to program it. Then we move on to the architecture of the SNES. Understanding the inner workings of the SNES is important to programming games for it. Once we have cleared the basics, I plan to continue with a guided game project. I will design and program a simple game clone for you to follow. That includes topics that go beyond basic programming. Things like sprite animation, collision detection, and input handling will be covered here. And finally, I want to show you how to design and produce your own PCB and cartridge to run your game on an actual SNES. To summarize, these articles will guide you through the complete process of creating your own SNES game.

I'm going to use tools that run on all major platforms and are free of charge. So you should be able to follow these articles whether you're on Linux, Windows, or OS X.

My goal is to keep these articles informative but short enough to be worked through within one hour (or less). I will try to keep each article as self-contained as possible. So people with some experience in programming NES, SNES, or other consoles may learn a trick or two. I aim to post a new article each weekend, but this may vary depending on the topic and scope. At the end of each article, you will find a list of references and links. They provide additional resources and pieces of information on topics covered in the article. If you have questions or suggestions, I'm always ready to help and welcome feedback. Use the comment function below this article.

The goal of this first article is simple: Set up a development environment and program your first SNES game (albeit a very, *very* boring one). Don't worry if you don't understand everything yet. I plan to build up knowledge gradually throughout the articles.

So, let's embark on an Assembly Adventure!



In the core of every beginning lives magic

For developing and testing SNES games, we need an array of tools. These are the four basic tools: A text editor, an assembler (and linker), an emulator, and git. Let's have closer look at them.

Programming without a proper **text editor** to edit source code is an exercise in futility. If you've programmed before, you probably have a preferred editor. For this series of articles, I will use the Atom Editor. Atom is an open source editor that supports a vast range of languages. It has thousands of plugins/packages to expand the basic functionalities of the editor. Later I will show you how to install Atom and the packages I use. Other good editors are Visual Studio Code, Sublime Text (costs $$$), and Vim. I prefer slim and fast editors over full-blown IDEs, but that is up to you. As long as you know how to edit source files with it, you will be fine.

Our second essential tool is an **assembler** (and linker). The assembler translates our source files into machine code that runs on the SNES. I will explain this process in-depth in future articles. We will use the cc65 toolchain to translate our code into machine code the SNES can execute. Several other assemblers for 6502/65816 exist, some geared towards NES and SNES development. But I prefer cc65 because it offers more flexibility than other assemblers.

Third, we need a **SNES emulator** to test our code. My emulator of choice is bsnes+. It offers some extra functionality useful for debugging code. I will show you how to use the bsnes+ debugger to find errors in your code. Other emulators like no$sns or 9snesx offer good debugging capabilities too. But bsnes+ works on all three major platforms without a hassle, so we'll use it.

Lastly, we need a source code management tool, **git**. We use git to download the source code for building some of the tools mentioned above. Git is a great tool for developers. You can find a lot of resources on the web about using git.

These are the basic tools we need for now. Later we will add some other tools for creating graphics, music, hex editor, etc.

Let's finally get started and install the tools.

Installing the tools

To make it easier to follow, I will use three icons to mark which instructions you should follow on your platform:

 **Linux**  **Windows**  **OS X**

Important note: There are multiple versions of bsnes. You will need to install bsnes+ (sometimes called bsnes-plus), bsnes+ adds debugging capabilities to bsnes, we will use these in later articles. Alternatively, you can download Mesen-SX from Github. You'll find build instructions for both below.

Windows users mind while reading the installation instructions that Linux and OS X use forward slashes `/` instead of backslashes `\` in (directory) path names.

Installation Shortcuts

Experienced users can skip the in-detail installation instructions and use these shortcuts instead.

Linux users use their distribution's package manager to install the following packages

```
cc65 atom git bsnes-plus
```

Atom is optional if you already have a text editor you use for programming. Git is probably already installed. The cc65 toolchain includes the assembler and linker we need to write SNES games. Mind that the exact package names may vary across distributions, make sure you get the correct ones.

🪟 Windows users need to install these programs:

- git
- Atom
- cc65
- mingw-w64 or MSYS2

Remember to add cc65 to your PATH. Alternatively, build cc65 from source as described here.

You can skip Atom if you have a preferred text editor.

You will still need to follow the in-detail instructions to build either the bsnes+ or Mesen-SX emulator below from source.

🍎 OS X users need to install the Xcode command line tools first. Open a terminal and enter

```
$ xcode-select --install
```

This will install among other tools a C++ compiler and git, which you need to build the bsnes+ emulator.

Next, install **Brew**. Homebrew is a Linux-style package manager that makes installations much easier. Open the link in the last sentence and execute the command in a terminal.

Once it is installed, use brew to install the following packages. Open a terminal and enter

```
$ brew install cc65
```

This will install the cc65 toolchain. Next, install Atom if you don't have a text editor for programming yet.

For building the bsnes+ emulator, please refer to the in-detail installation instructions below.

🐧🪟🍎 Check your installation by opening a terminal and running these commands:

```
$ git --version
$ cc65 --version
$ bsnes
```

If you decide to use Atom, read the in-detail installation instructions for suggested additional packages that will enhance the editor. If you get stuck, try the in-detail installation instructions below or send me a message. The linked homepages for each tool may be helpful too.

Detailed Installation Instructions

git

Start by installing **git**.

🐧 Most Linux distributions ship with git installed. Else, use your distribution's package manager to install git. For Debian, Ubuntu, and the like open a terminal and type

```
$ apt-get install git
```

🪟 Windows users download the git installer and execute it.

 For OS X users the simplest way to install git is to install the Xcode command line tool as described above. Open a terminal and type the command

```
$ xcode-select --install
```

This will install a C++ toolchain you need to build bsnes+.

🐧🪟 Test your git installation by opening a terminal and typing

```
$ git --version
```

Next, **the editor**. Installing it is as easy as it gets. If you have a preferred editor, you can skip this step.

Atom

🐧 If you're on Linux, open a terminal and use your package manager to install Atom:

```
$ apt-get install atom
```

Alternatively, download the .deb or .rpm file and install it.

🪟 🍎 Windows and Mac users can download the appropriate Windows or OS X installer from the Atom website and install it.

🐧 🪟 🍎 You're pretty much done with this step. I suggest you install at least two extra packages for Atom. The first is language-65asm which will add syntax highlighting for 6502/65816 assembly. I also recommend vim-mode-plus. If you're not familiar with vim or emacs, read this. They greatly improve productivity for programmers, so choose one and learn to use it. Take some time to familiarize yourself with Atom by reading the Atom Flight Manual. Choose a UI and syntax theme you like, and once you're happy with your editor, come back here.

Next, is the **cc65 toolchain**.

CC65

This step is easiest on Linux and OS X, Windows users will need to take an extra step.

🐧 Linux users again use their trusty package manager to install cc65:

```
$ apt-get install cc65
```

If you prefer to build from source, follow the instructions of the cc65 GitHub page.

🪟 Windows users download and extract the most recent snapshot of cc65, place it in a location of your choice(I suggest `C:\cc65` for simplicity). Don't forget to add `C:\cc65\bin` to your PATH.

🍎 OS X users will enlist the help of their (new) package manager, brew. Open a terminal and type

```
$ brew install cc65
```

🐧 🪟 🍎 Test your installation by opening a terminal and typing:

```
$ cc65 --version
```

If you run into any trouble, check out the cc65 Wiki on GitHub for installation instructions. They have detailed installation instructions for each system.

Next, time to build an emulator. You can use either bsnes+ or Mesen-SX.

BSNES+

🐧 Linux users use their package manager to install bsnes+:

```
$ apt-get install bsnes-plus
```

Unfortunately, for Windows and Mac users the process is a bit more complicated. You will have to build the emulator from source. I'll try and guide you through every single step.

🪟 Windows users need to take three steps:

- Install mingw-w64 or msys2
- Get bsnes+ source code
- Build bsnes+

Installing mingw-w64 is easy, download the Installer and execute it. Test it by opening a command line window and typing

```
$ g++
```

If Windows can't find the application, you need to add `MinGW\bin` to your path the same way you added `C:\cc65\bin`.

Alternatively, install MSYS2 from here.

Now download the bsnes+ source code with git from GitHub. Open a command line window and navigate to a directory where you want to keep bsnes+, e.g. `C:\bsnesplus`. Once you're there, download the source code of bsnes+ with git:

```
$ mkdir bsnesplus
$ cd bsnesplus
```

```
$ git clone --recurse-submodule https://github.com/devinacker/bsnes-plu
```

This will create a new directory called `bsnesplus`. Navigate to it and start the build process:

```
$ cd bsnesplus\bsnes
$ make
```

If no error occurs during building, you should find the bsnes+ executable in the `bin` subdirectory. If you have trouble building bsnes+, check the bsnes+ GitHub page.

 bsnes+ support is unfortunately shaky. Check out this fork of bsnes+ for build instructions on OSX.

Mesen-SX

 You're going to need a current installation of Visual Studio Community Edition to build Mesen-SX (I've tested it with 2019). Then just follow the build instructions described in the Mesen-SX repository:

1. Open the solution in Visual Studio 2019
2. Set "UI" as the Startup Project
3. Compile as Release/x64
4. Run the project from Visual Studio (or start it in the file explorer)

 Use your package manager to install your distributions equivalents of the packages `mono-devel` and `libsdl2-dev`. Then, clone the repository and build it using make:

```
$ sudo apt-get install mono-devel libsdl2-dev
$ git clone https://github.com/NovaSquirrel/Mesen-SX.git
$ cd Mesen-SX
$ make
```

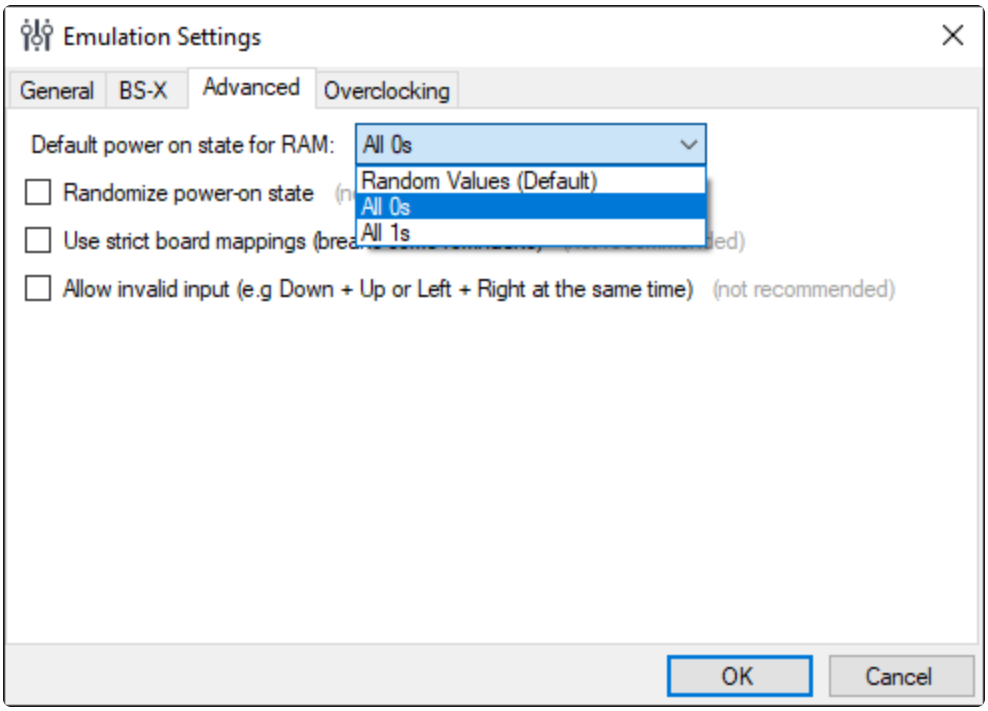
If nothing went wrong you now have git, an editor, the cc65 toolchain, and an emulator working on your system. Good work!

Test Your Emulator

Test your emulator by starting it (choose compatibility profile for bsnes+; we will talk about the difference between the compatibility and accuracy profile in some later articles) and running this test ROM I have written. Got the message? Excellent.

To finish off this article let's move on and write your very first SNES game.

IMPORTANT: If you're using Mesen-S(X) make sure you set "Default power on state for RAM:" in the emulation settings to "All 0s":



Mesen-S(X) Emulation Settings

Otherwise, the code examples in the next few articles won't work. We will remedy that in a future article where we learn to properly initialize the SNES.

Your First Game

You will now write your first lines of 65816 assembly code. I suggest you keep all files in one directory. For this article, I will store all files in `~/assemblyadventure/lesson01` on my system. Windows users might use `C:\assemblyadventure\lesson01`.

Open your editor and create a new file `nihil.s` and enter this code:

```
1  ; ------------------------------------------------------------------------
2  ;   File: nihil.s
3  ;   Description: Your very first SNES game!
4  ; ------------------------------------------------------------------------
```

```
5
6   ;----- Assembler Directives ---------------------------------------------------
7   .p816                           ; this is 65816 code
8   .i16                            ; X and Y registers are 16 bit
9   .a8                             ; A register is 8 bit
10  ;------------------------------------------------------------------------------
11
12  ;------------------------------------------------------------------------------
13  ;   This is were the magic happens
14  ;------------------------------------------------------------------------------
15  .segment "CODE"
16  .proc   ResetHandler            ; program entry point
17          sei                     ; disable interrupts
18          clc                     ; clear the carry flag...
19          xce                     ; ...and switch to native mode
20
21          lda #$81                ; enable...
22          sta $4200               ; ...non-maskable interrupt
23
24          jmp GameLoop            ; initialisation done, jump to game loop
25  .endproc
26
27  .proc   GameLoop                ; The main game loop
28          wai                     ; wait for NMI interrupt
29          jmp GameLoop            ; jump to beginning of main game loop
30  .endproc
31
32  .proc   NMIHandler              ; NMIHandler, called every frame/V-blank
33          lda $4210               ; read NMI status
34          rti                     ; interrupt done, return to main game loop
35  .endproc
36
37  ;------------------------------------------------------------------------------
38  ;   Interrupt and Reset vectors for the 65816 CPU
39  ;------------------------------------------------------------------------------
40  .segment "VECTOR"
41  ; native mode   COP,        BRK,        ABT,
42  .addr           $0000,      $0000,      $0000
43  ;               NMI,        RST,        IRQ
44  .addr           NMIHandler, $0000,      $0000
45
46  .word           $0000, $0000    ; four unused bytes
47
48  ; emulation m.  COP,        BRK,        ABT,
```

```
49  .addr           $0000,      $0000,      $0000
50  ;               NMI,        RST,        IRQ
```

nihil.asm hosted with ❤ by **GitHub** **view raw**

Next, create another file called `memmap.cfg`, enter this code:

```
1   # Memory Map for a minimal SNES game
2
3   MEMORY
4   {
5       ROM0: start = $008000, size = $8000, fill = yes;
6   }
7
8   SEGMENTS
9   {
10      CODE:   load = ROM0, align = $100;
11      VECTOR: load = ROM0, start = $00ffe4;
12  }
```

memmap.cfg hosted with ❤ by **GitHub** **view raw**

Don't forget to save. Next, you will build your very first SNES game! In case you're wondering about the different file extensions, I prefer to use `.s` for assembly files, but `.asm` is fine too. I use `.asm` for the code Gists as GitHub will use different syntax highlighting depending on the file extension used.

Open a terminal and navigate to the directory where the files reside you just created. Then enter the following command to invoke the cc65 toolchain to build your first game:

```
$ ca65 --cpu 65816 -o nihil.o nihil.s
$ ld65 -C memmap.cfg nihil.o -o nihil.smc
```

Make sure you use an upper-case C as an argument on the second command. Otherwise, ld65 will complain. You might wonder about ca65 and ld65. Those are the assembler and linker I told you earlier about. The cc65 toolchain is made up of several tools. We will take a closer look at how they work in the future when appropriate. If everything worked, there should be a new file called `nihil.smc`.

Start your emulator and load your new file with it. On startup, select the compatibility profile. Now you should see this:



Your first SNES game!

Congratulations! This is your first SNES game. It is very nihilistic since it does absolutely *nothing*. (That's not entirely true, the code acknowledges that the so-called Non-Maskable Interrupt occurred - and returns to waiting/doing nothing. But more on this when we talk about the SNES architecture). I told you your first game would be boring.

But don't be disappointed, you have achieved a lot today. You have set up a development environment. This is often the hardest part of getting your code to run on a new platform. You will use your new environment to learn all about SNES assembly programming. Your adventure has just started.

That's all for today. In the next article, I will introduce you to 65816 assembly programming. I hope you enjoyed this first article of hopefully many more to come.

If you wanna look at some actual SNES code, I have written a simple demo that shows some basic functions. Follow the build instructions provided in the repository README. Try to build

this demo by yourself to familiarize yourself with the build process. We will use makefiles in future articles to speed up development.

I'm always happy for comments and suggestions, please use the comment section below.

References and Links

- Here is a nice article about the history of the SNES
- If you prefer watching videos check this one out. Watch his other videos too, they're really good!
- The Super Famicom Development Wiki is a great resource for SNES programming. I will reference it in future articles, so add it to your bookmarks
- All SNES Assembly Adventure code examples of this series on Github

Critique me

Since this is my first article, I'd really like to hear what you think about it. Please let me know in the comments what you liked and disliked, what you think I missed or got wrong, or just your general opinion. Thank you!

ALSO ON MACHINE CODE CONSTRUCTION YARD

SNES Assembly Adventure 03: …
8 years ago · 22 comments
I do assembly programming on old machines and boat building.

SNES Assembly Adventure 09: Project …
4 years ago · 1 comment
I do assembly programming on old machines and boat building.

Blog Comment Policy
8 years ago · 1 comment
I do assembly programming on old machines and boat building.

A CMake Ter
Game Boy De
3 years ago · 1 c
I do assembly
on old machine
building.